Лабораторная работа №2.

Динамическое программирование на битовых масках

Цель работы

Изучить технику динамического программирования на битовых масках для решения задач комбинаторной оптимизации на примере задачи коммивояжера. Научиться реализовывать точный алгоритм для малых размерностей графа ($N \leq 15$-20), освоить понятие "маска состояния", научиться восстанавливать оптимальный маршрут.

Теоретическая часть

1. Постановка задачи

Задача коммивояжера — одна из классических задач комбинаторной оптимизации. Формулируется она следующим образом:

Имеется **N** городов, пронумерованных от 0 до N-1. Задана матрица расстояний **dist[i][j]** — стоимость (расстояние, время, цена) переезда из города i в город j. Требуется найти кратчайший замкнутый маршрут (цикл), который начинается в некотором городе (обычно в городе 0), проходит через все остальные города ровно по одному разу и возвращается в исходный город.

2. Динамическое программирование с битовыми масками

Ключевая идея метода заключается в том, что оптимальное решение задачи может быть построено из оптимальных решений подзадач. Подзадача может быть определена следующим образом: мы уже посетили некоторое подмножество городов **S** и закончили маршрут в некотором городе $i \in S$. Нам нужно знать минимальную стоимость такого частичного маршрута, чтобы на его основе строить полный маршрут.

Проблема: как компактно хранить информацию о том, какие города уже посещены? Здесь на помощь приходят битовые маски.

Битовая маска — это целое число, каждый бит которого соответствует одному элементу множества. Если бит равен 1, элемент присутствует; если 0 — отсутствует.

Для **N** городов мы можем использовать число от 0 до (2^N - 1), где:

- Бит 0 (младший бит) соответствует городу 0
- Бит 1 соответствует городу 1
- ...
- Бит N-1 соответствует городу N-1

Пример для N = 4:

- Маска 0001 (двоичная) = 1 (десятичная): посещен только город 0
- Маска 0011 = 3: посещены города 0 и 1
- Маска 1010 = 10: посещены города 1 и 3
- Маска 1111 = 15: посещены все 4 города

3. Рекуррентное соотношение
Определим состояние динамического программирования:

dp[mask][i] — минимальная стоимость маршрута, который начинается в стартовом городе (обычно 0), посещает все города, входящие в множество **mask**, и заканчивается в городе **i**. При этом город **i** обязательно должен принадлежать множеству **mask**.

База динамического программирования:
dp[1 << 0][0] = 0 — маршрут из города 0, который посетил только город 0 (сам себя) и закончился в городе 0, имеет стоимость 0. Все остальные состояния инициализируются бесконечностью.

Переходы:
Из состояния **(mask, i)** мы можем попытаться пойти в любой еще не посещенный город **j** (такой, что бит j в mask равен 0). Новая маска будет
new_mask = mask | (1 << j). Стоимость нового маршрута будет равна стоимости текущего маршрута плюс расстояние от i до j:
dp[new_mask][j] = min(dp[new_mask][j], dp[mask][i] + dist[i][j])

Финальный ответ:
После того как мы посетили все города (маска = (1 << N) - 1), нам нужно вернуться в начальный город (город 0). Для каждого конечного города i (кроме самого 0) мы добавляем расстояние от i до 0 и берем минимум:
answer = min(dp[(1 << N) - 1][i] + dist[i][0]) для всех i от 1 до N-1

4. Восстановление маршрута
Для восстановления самого маршрута (последовательности городов) необходимо хранить дополнительный массив **parent[mask][i]**, в котором записывается, из какого предыдущего города мы пришли в состояние **(mask, i)**. После нахождения оптимальной стоимости мы можем пройти по этим ссылкам назад и восстановить полный путь

Требования к реализации (общие для всех вариантов)

1. **Ввод данных:**
   - Для вариантов 1-4: сначала число N, затем матрица N×N (целые числа).
   - Для варианта 5: сначала число N, затем N строк с координатами x y (целые или вещественные).
   - Для варианта 4 дополнительно вводятся start и end.
2. **Вывод:**
   - Первая строка — минимальная стоимость (для варианта 5 — вещественное число с двумя знаками после запятой).
   - Вторая строка — последовательность городов через пробел (включая начальный и конечный согласно варианту).
3. **Ограничения:**
   - $N \leq 15$
4. **Дополнительные требования:**
   - Реализовать восстановление маршрута (не только стоимость).
   - Использовать битовые маски для хранения состояний.
   - Обеспечить защиту от дурака: проверка корректности ввода, обработка отрицательных расстояний (если разрешены).

Индивидуальные варианты

Вариант 1. Классический (симметричный)
Цель: Найти минимальную стоимость гамильтонова цикла, начинающегося и заканчивающегося в городе 0.
Особенность: Граф симметричный (неориентированный), то есть dist[i][j] = dist[j][i].
Вход: Матрица расстояний N×N.
Выход: Минимальная стоимость и последовательность городов.

Вариант 2. Асимметричный
Цель: Найти минимальную стоимость гамильтонова цикла в ориентированном графе.
Особенность: dist[i][j] может не равняться dist[j][i]. Например, из-за одностороннего движения или рельефа местности.
Вход: Матрица расстояний N×N (не обязательно симметричная).
Выход: Минимальная стоимость и последовательность городов.

Вариант 3. Гамильтонов путь (без возврата)
Цель: Найти минимальную стоимость пути, который начинается в городе 0, проходит через все остальные города ровно один раз, но НЕ обязан возвращаться в начало.
Отличие от классики: Не нужно добавлять расстояние от последнего города до начального.
Вход: Матрица расстояний N×N.
Выход: Минимальная стоимость пути и последовательность городов.

Вариант 4. С фиксированным началом и концом
Цель: Найти минимальную стоимость пути, который начинается в городе start и заканчивается в городе end, проходя через все остальные города ровно один раз.
Особенность: Начальный и конечный города задаются отдельно и могут быть разными.
Вход: Матрица расстояний N×N, номера start и end.
Выход: Минимальная стоимость и последовательность городов.

Вариант 5. На координатной плоскости (евклидово расстояние)
Цель: Найти минимальный гамильтонов цикл для городов, заданных координатами на плоскости.
Особенность: Расстояние вычисляется по формуле евклидова расстояния: $\sqrt{(x_1-x_2)^2 + (y_1-y_2)^2}$. Матрица не дается, нужно вычислить самостоятельно.
Вход: Количество городов N, затем N строк с координатами x y.
Выход: Минимальная длина маршрута (вещественное число) и последовательность городов.

Тесты для проверки

Вариант 1:

Вход	Вывод
2 0 5 5 0	10 0 1 0
3 0 10 15 10 0 20 15 20 0	45 0 1 2 0
4 0 10 15 20 10 0 35 25 15 35 0 30 20 25 30 0	80 0 1 3 2 0

Вариант 2:

Вход	Вывод
2 0 5 3 0	8 0 1 0
3 0 1 5 2 0 1 3 4 0	5 0 1 2 0
4 0 2 3 4 5 0 6 7 8 9 0 1 2 3 4 0	11 0 1 2 3 0

Вариант 3:

Вход	Вывод
2 0 5 5 0	5 0 1
3 0 10 15 10 0 20 15 20 0	30 0 1 2
4 0 10 15 20 10 0 35 25 15 35 0 30 20 25 30 0	65 0 1 3 2

Вариант 4:

Вход	Вывод
3 0 10 15 10 0 20 15 20 0 0 2	30 0 1 2
3 0 10 15 10 0 20 15 20 0 1 0	35 1 2 0
4 0 10 15 20 10 0 35 25 15 35 0 30 20 25 30 0 0 3	65 0 1 2 3

Вариант 5:

Вход	Вывод
2 0 0 3 4	10.00 0 1 0
3 0 0 3 0 0 4	12.00 0 1 2 0
4 0 0 1 0 1 1 0 1	4.00 0 1 2 3 0